Manual de instalación — SmartHome

Servicio Nacional de Aprendizaje (SENA)
Programa: Análisis y Desarrollo de Software · Ficha: 3145555
Proyecto: SmartHome · Revisión documental: 7 de noviembre de 2026
Equipo: Karen Daniela Holguín Cruz, Natalia Chala Chala y Kevin Stiven López Amaya
Estado: versión preparada para revisión académica; no publicada en el PR #17 ni aprobada por el instructor.

> Control de versión. Estos manuales parten de los tres Markdown de la rama final-delivery/smarthome-stiven-20261107 del PR institucional #17 y se contrastaron con README y manifiestos package.json de las ramas dev accesibles el 7-11-2026. Son una referencia documental de desarrollo, no certificación de funcionamiento de una versión main etiquetada. No se ejecutaron instalaciones, pruebas Android ni operaciones eléctricas durante su elaboración.

1. Alcance y requisitos

Este procedimiento corresponde a instalación local en Windows PowerShell para desarrollo o evaluación. No habilita despliegues de producción. No se ejecutaron estos comandos en un equipo del usuario durante la elaboración; cada responsable debe registrar capturas/logs, commit y resultado. El equipo debe aprobar las credenciales, permisos y entorno de destino.

Tabla 1

Alcance y requisitos

Herramienta	Referencia	Propósito
Git	Versión funcional reciente	Clonar repositorios
Node.js y npm	Node 22 utilizado en CI de backend/web	Ejecutar NestJS, Vite y Expo
Docker + Compose	Compose v2	PostgreSQL y Liquibase
Android / Expo	Equipo autorizado; build nativa para BLE	Validación móvil
PowerShell	Windows 10/11	Comandos de ejemplo

Fuente: elaboración propia.

Precaución de versiones: from_movil/dev/package.json muestra Expo ~58.0.23 y React Native 0.88.1; su README continúa describiendo Expo 57. Antes de compilar, comprobar package-lock.json, eas.json, CI y la rama real aprobada. No forzar migraciones de SDK por estas instrucciones.

Como marco de referencia, el manual sigue la organización por tareas de ISO/IEC/IEEE 26514 (ISO/IEC/IEEE, 2022); la instalación usa Docker Compose (Docker Inc., s. f.) y las migraciones de Liquibase (Liquibase, s. f.).

2. Obtener los cuatro componentes

En una carpeta de trabajo nueva y autorizada:

```
New-Item -ItemType Directory -Force SmartHome | Out-Null
Set-Location SmartHome
git clone https://github.com/nataliachala1/smarthome-database.git
git clone https://github.com/nataliachala1/smarthome-backend.git
git clone https://github.com/nataliachala1/smarthome-frontend-web.git
git clone https://github.com/Kevin-Stiven-Lopez-Amaya404/from_movil.git
```

En cada repositorio, ejecutar git branch -a y git log -1 --oneline. El responsable debe indicar la rama y el SHA que serán evaluados. No mezclar dev, qa y main sin validar compatibilidad; no asumir que el tag de la carpeta institucional corresponde a esos commits.

3. PostgreSQL y migraciones

Comenzar con una base local aislada. El docker-compose.yml del repositorio de BD utiliza PostgreSQL 16 Alpine en el entorno de desarrollo documentado. Proteger los volúmenes; down -v puede borrar los datos.

```
Set-Location smarthome-database
Copy-Item .env.example .env
# Editar .env con valores LOCALES y una contraseña nueva.
docker compose config
docker compose up -d --wait postgres
docker compose ps
```

Antes de aplicar cambios, comprobar el objetivo de conexión y revisar el SQL generado:

```
docker compose run --rm liquibase --defaults-file=liquibase.properties validate
docker compose run --rm liquibase --defaults-file=liquibase.properties update-sql
```

Solo si es una base local desechable, aprobada y respaldada cuando corresponda:

```
# Modifica la base de datos configurada: requiere autorizacion.
docker compose run --rm liquibase --defaults-file=liquibase.properties update
```

No usar credenciales propietarias de migración como credenciales normales del backend. Revisar aprovisionamiento de roles LOGIN y políticas RLS siguiendo el README de BD. Nunca publicar .env ni capturas de contraseñas.

4. API NestJS

Desde la carpeta SmartHome creada en el paso 2:

```
Set-Location ..\smarthome-backend
Copy-Item .env.example .env
npm ci
npm run start:dev
```

npm ci puede ejecutar prisma generate según el package.json del backend. Configurar valores locales reales de DATABASE_URL, DATABASE_IOT_URL, secretos JWT, PORT, CORS y opciones MQTT; no copiar contraseñas de documentos históricos. La API utiliza /api/v1 y

Swagger puede aparecer en /api/docs únicamente si está habilitado. Confirmar arranque y lectura de una ruta pública autorizada sin publicar tokens.

Tabla 2

API NestJS

Grupo de variables	Uso
DATABASE_URL, DATABASE_IOT_URL	Cuentas limitadas de PostgreSQL para aplicación e ingesta
JWT_SECRET, JWT_REFRESH_SECRET	Sesiones y tokens; mantener secretos fuera de Git
FRONTEND_URL, CORS_ALLOWED_ORIGINS	Orígenes admitidos en web
MQTT_ENABLED, MQTT_BROKER_URL, SHELLY_*	Integración IoT sujeta a entorno seguro

Fuente: elaboración propia.

5. Cliente web React y Vite

Ejecutar desde web/, no desde la raíz del frontend:

```
Set-Location ..\smarthome-frontend-web\web
Copy-Item .env.example .env.local
npm ci
npm run dev
```

En web/.env.local, VITE_BACKEND_URL indica el servidor de destino del proxy de desarrollo (ejemplo local: http://localhost:3000); VITE_API_URL suele ser /api/v1. La consola indica el puerto Vite (habitualmente 5173). Verificar /login y las solicitudes de la API. Las variables VITE_* no son secretas: pueden incluirse en el bundle del navegador.

El docker-compose.yml de la raíz web hace referencia a carpetas externas; no intentar ejecutar toda la pila desde un clon aislado sin preparar esas rutas. El README web documenta esta limitación.

6. Aplicación móvil Expo para Android

Desde la carpeta de proyecto:

```
Set-Location ..\..\from_movil
Copy-Item .env.example .env.local
npm ci
npm run lint
npm run typecheck
npx expo start --clear --lan
```

Definir EXPO_PUBLIC_API_URL con la URL completa accesible desde el teléfono, terminada en /api/v1. Si la API funciona en el PC y el teléfono está en la misma LAN, utilizar la IP LAN real del PC; localhost en Android es el propio teléfono. Revisar Wi-Fi y firewall de manera autorizada.

Las funciones BLE requieren módulos nativos y pueden no funcionar en Expo Go. Emplear una Development Build o APK creada para el mismo SDK y commit que se está evaluando. No ejecutar comandos EAS de publicación OTA sin revisión, porque pueden actualizar una instalación existente.

7. Comprobaciones antes de evaluar

Tabla 3

Comprobaciones antes de evaluar

Componente	Comandos orientativos	Evidencia necesaria
API	npm run lint, npm test, npm run build	Ejecución, SHA y resultado
Web en web/	npm run lint, npm run typecheck, npm run test:contracts, npm run build	Logs de build y consulta API
DB	liquibase validate y update-sql	Destino y changelog validados
Móvil	npm run lint, npm run typecheck, npm run test:mobile-config, npm run test:navigation-auth	Suite y compilación Android cuando aplique

Fuente: elaboración propia.

Los resultados CI históricos que figuran en el manual original no prueban funcionamiento del commit final, ni autorizan declarar seguro el APK. Registrar también una prueba real de inicio de sesión, consulta de hogar y, si se cuenta con dispositivo autorizado, telemetría Shelly.

8. Solución de problemas

Tabla 4

Solución de problemas

Problema	Comprobación
API no inicia	.env, PostgreSQL, puerto y permisos LOGIN/RLS
Error de migración	validate, update-sql, rol y backup, sin destruir datos
Web sin datos	VITE_BACKEND_URL, VITE_API_URL, proxy y CORS
Android sin API	EXPO_PUBLIC_API_URL, LAN, firewall y puerto
BLE no disponible	Verificar build nativa, versión de Expo y permisos
Orden Shelly sin resultado	Contrastar respuesta API, MQTT y estado físico

Fuente: elaboración propia.

9. Entrega y fuentes

Se entrega un procedimiento reproducible para ejecutar después con permiso, no una constancia de instalación exitosa. Antes del tag final, adjuntar acta de instalación, commit, ambiente, comandos realmente ejecutados y resultados.

- Manual de instalación del PR #17
- README de BD
- README backend
- README web
- README móvil

Anexo A. Registro de validación para completar durante la instalación

La siguiente tabla no representa pruebas ejecutadas. El responsable puede diligenciarla después de trabajar en un entorno autorizado y adjuntar logs o capturas depuradas.

Tabla 5

Anexo A. Registro de validación para completar durante la instalación

Paso	Commit o referencia	Resultado observado	Evidencia
Clonación de repositorios	Por registrar	Pendiente	Pendiente
PostgreSQL saludable	Por registrar	Pendiente	Pendiente
Liquibase validate/update-sql	Por registrar	Pendiente	Pendiente
Backend y API	Por registrar	Pendiente	Pendiente
Login web contra API	Por registrar	Pendiente	Pendiente
Acceso Android a API	Por registrar	Pendiente	Pendiente
Prueba Shelly física (si aplica)	Por registrar	Pendiente	Pendiente

Fuente: elaboración propia.

Criterio de cierre: registrar fecha, entorno, versiones exactas, resultados y responsable, sin incorporar datos personales, tokens ni contraseñas en un repositorio público.

Referencias

Docker Inc. (s. f.). *Docker Compose documentation*. Recuperado el 8 de noviembre de 2026, de https://docs.docker.com/compose/

ISO/IEC/IEEE. (2022). *Systems and software engineering — Design and development of information for users* (ISO/IEC/IEEE 26514:2022). https://www.iso.org/standard/77451.html

Liquibase. (s. f.). *Liquibase documentation*. Recuperado el 8 de noviembre de 2026, de https://docs.liquibase.com/